

ЛЕКЦИЯ 2

Концепции и архитектура на БД



5th Edition

Elmasri / Navathe

Структура на лекцията

- Модели на данните и категориите им
- История на моделите на данни
- Схеми, инстанции и състояния
- Three-Schema Architecture
- Независимост на данните
- DBMS езици и интерфейси
- Database System Utilities and Tools
- Централизирани и клиент-сървър архитектури
- Класификация на DBMS

Модели на данните

- **Data Model:**

- Множество от концепции, което описва **структура** на БД, **операциите** за обработка на тези структури и редица **ограничения**, които БД трябва да спазва.

- **Data Model Structure and Constraints:**

- Конструкциите се използват за дефиниране на структури в БД
- Обикновено конструкциите включват **елементи** (и техните **типове данни**), групи от елементи (т.е. **entity, record, table**) и **връзки** между тези групи
- Ограниченията задават някои рестрикции върху валидността на данните; те трябва да са в сила по всяко време

Модели на данните (продължение)

- **Data Model Operations:**

- Тези операции се ползват за задаване на *извличане* и *актуализации* върху БД, чрез рефериране на конструкциите в модела на данните.
- Операциите върху данновия модел може да включват ***basic model operations*** (т.е. insert, delete, update) и ***user-defined operations*** (т.е. съхранени процедури)

Категории модели данни

- **Conceptual (high-level, semantic) data models:**
 - Предоставят концепции, които са свързани с представянето на данните според потребителите.
 - (наричат се *entity-based* или *object-based* data models.)
- **Physical (low-level, internal) data models:**
 - Предоставят концепции, които описват в детайли как данните са съхранени в компютъра. Те се задават чрез дизайна и администрирането на DBMS
- **Implementation (representational) data models:**
 - Предоставят концепции, които се намират между предходните 2 модела и се ползват от комерсиалните DBMS реализации.

Схеми срещу инстанции

- Database Schema:
 - **Описание** на БД.
 - Включва описание на структурите на БД, типовете данни и ограниченията върху БД.
- Schema Diagram:
 - **Илюстративен** поглед на схемата на БД.
- Schema Construct:
 - **Компонент** на схемата или обект в схемата, т.е. STUDENT, COURSE.

Схеми срещу инстанции

- Database State:
 - Данните, съхранени в БД в **даден момент от време**. Тук се включват всички данни в БД.
 - Нарича се още database instance.
 - Терминът *instance* се прилага и към индивидуални компоненти на БД, т.е. *record instance*, *table instance*, *entity instance*

Database Schema срещу Database State

- Database State:
 - Отнася се до **съдържанието** на БД в даден момент от време.
- Initial Database State:
 - Отнася се до състоянието на БД, което се зарежда в началото.
- Valid State:
 - Състояние, което удовлетворява структурата и ограниченията на БД.

Database Schema срещу Database State (продължение)

- Разлика
 - *database schema* не се променя често.
 - *database state* се променя с промяна на данните в БД.
- **Schema** се нарича и **intension**.
- **State** е познато и кат **extension**.

Пример на Database Schema

STUDENT

Name	Student_number	Class	Major
------	----------------	-------	-------

COURSE

Course_name	Course_number	Credit_hours	Department
-------------	---------------	--------------	------------

PREREQUISITE

Course_number	Prerequisite_number
---------------	---------------------

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
--------------------	---------------	----------	------	------------

GRADE_REPORT

Student_number	Section_identifier	Grade
----------------	--------------------	-------

Figure 2.1

Schema diagram for the database in Figure 1.2.

Пример на database state

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2

A database that stores student and course information.

Three-Schema Architecture

- Поддържа следните DBMS характеристики:
 - **Независимост на данните от програмата.**
 - Поддръжка на **множество изгледи** на данните.
- Не се използва явно в комерсиалните DBMS продукти.

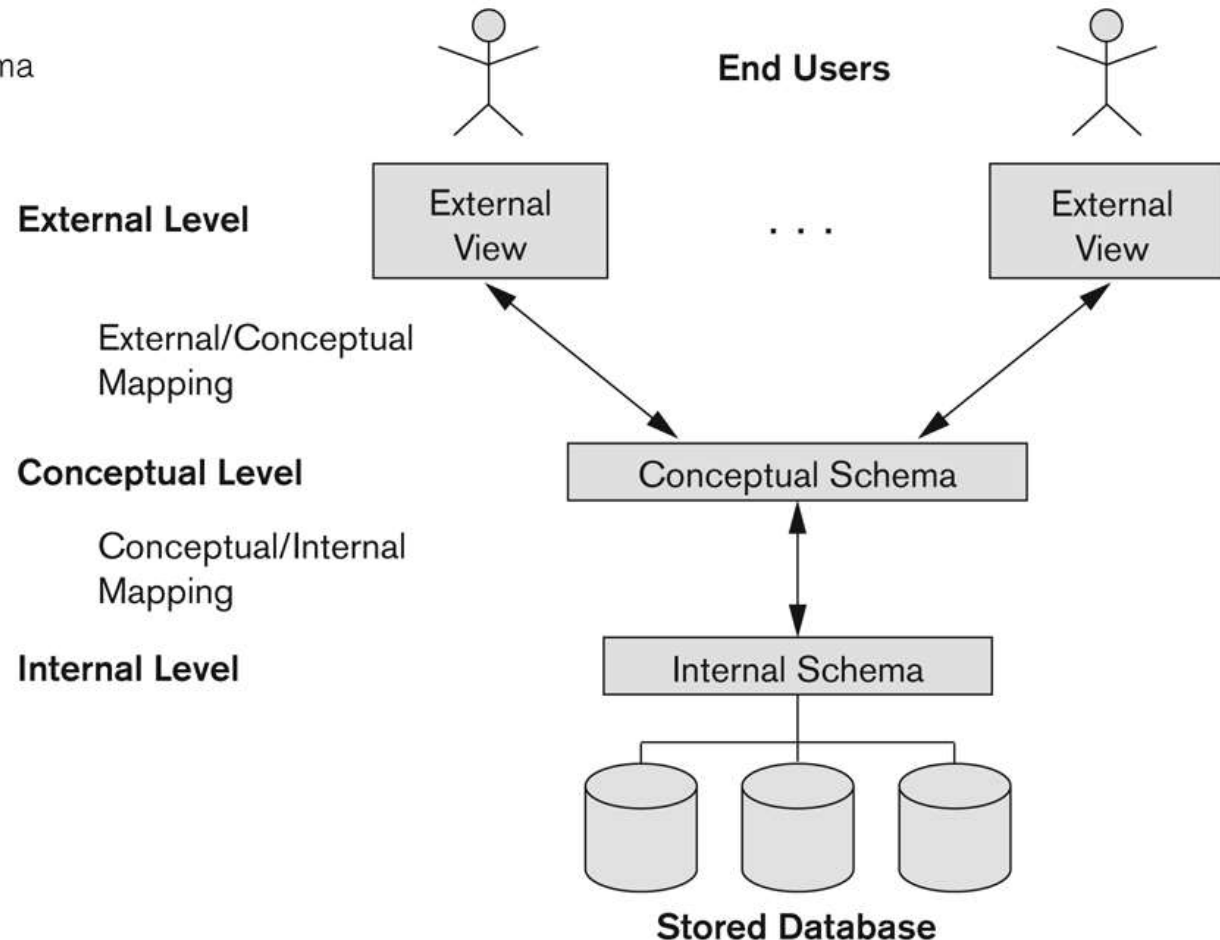
Three-Schema Architecture

- Дефинира DBMS схеми на *три* нива:
 - **Internal schema** на вътрешно ниво за описание на физическото съхранение на структурите и достъпа до тях (т.е. indexes).
 - Обикновено използва **физически** модел на данните.
 - **Conceptual schema** на концептуално ниво описва структурите и ограниченията за цялата БД.
 - Използва **концептуален** или **implementation** модел на данните.
 - **External schemas** на външно ниво за описание на различни изгледи за потребителите.
 - Обикновено използва същия даннов модел, както концептуалната схема.

The three-schema architecture

Figure 2.2

The three-schema architecture.



Three-Schema Architecture

- Mappings между схемите на различни нива е необходим за трансформиране на заявките и данните.
 - Програмите използват външна схема и съпоставяне от DBMS на вътрешна схема за изпълнение.
 - Извлечените данни от вътрешно ниво от DBMS се преформатират, за да се представят според изискванията на външния изглед (напр. Форматиране на резултатите от SQL заявка за показване в Web page)

Независимост на данните

- **Logical Data Independence:**

- Възможността да се смени концептуалната схема без промени във външната и свързаните с нея приложения.

- **Physical Data Independence:**

- Възможността да се смени вътрешната схема без промени по концептуалната.
- Пример: вътрешната схема може да се промени при реорганизация на файловата структура или създаване на нови индекси, за подобряване на производителността на БД.

Независимост на данните (продължение)

- Когато се промени схемата на ниско ниво, трябва да се промени само **mappings** между тази схема и схемите на по-високо ниво в DBMS, за да се поддържа даннова независимост.
- Схемите на по-високо ниво **не се променят**.
 - Следователно, приложната програма не трябва да се променя, тъй като използва външните схеми.

DBMS езици

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
 - Високо ниво или непроцедурни езици: тук се включва релационния език SQL
 - Може да се ползва самостоятелно или вграден в програмен код
 - Ниско ниво или процедурни езици:
 - Те се вграждат в програмен код

DBMS езици

- **Data Definition Language (DDL):**
 - Използва се от DBA и дизайнерите на БД за задаване на концептуалната схема на БД.
 - В много DBMS, DDL се ползва и за дефиниране на вътрешната и външната схеми (изгледи).
 - В някои DBMS, се ползват отделни езици за дефиниране на вътрешната и външната схеми: **storage definition language (SDL)** и **view definition language (VDL)**.
 - Обикновено SDL е реализиран през DBMS команди, предоставени на DBA и дизайнерите на БД

DBMS езици

- **Data Manipulation Language (DML):**
 - Използва се за задаване на актуализации и изличане на данни от БД
 - Командите DML (data sublanguage) могат да се *вграждат* в езици за програмиране с общо предназначение (host language), като COBOL, C, C++ или Java.
 - За достъп до DBMS от език за програмиране се предоставя и библиотека от функции.
 - Самостоятелните DML команди могат да се полват директно (*query language*).

Типове DML

- **High Level или Non-procedural Language:**
 - Пример: SQL relational language
 - Те са множествоно ориентирани и задават какви данни са извлечени, а не как.
 - Декларативни езици.
- **Low Level или Procedural Language:**
 - Извличат само по един запис в даден момент от време;
 - За получаване на повече записи се ползват цикли и позициониране с указатели.

DBMS интерфейси

- Самостоятелни query language интерфейси
 - Пример: Интерактивни SQL заявки в (т.е. SQL*Plus в ORACLE)
- Програмен интерфейс за вграждане на DML в език за програмиране
- Дружелюбен интерфейс
 - Меню базиран, базиран на форми, графичен и т.н.

DBMS интерфейс за програмни езици

- Програмни интерфейси за вграждане на DML в езици за програмиране:
 - **Embedded Approach:** embedded SQL (за C, C++ и др.), SQLJ (за Java)
 - **Procedure Call Approach:** JDBC за Java, ODBC за другите езици за програмиране
 - **Database Programming Language Approach:** ORACLE има PL/SQL - програмен език базиран на SQL; езиците вграждат SQL и неговите типове данни като компоненти

Database System Utilities

- Изпълняват редица функции като:
 - Зареждане на данни, съхранени в БД, включително необходимото конвертиране.
 - Backing up на БД периодично.
 - Реорганизация на файловата структура на БД.
 - Генериране на отчети.
 - Наблюдение на производителността на БД.
 - Други функции като сортиране, компресия на данните и т.н.

Други Tools

- Data dictionary / repository:
 - Използва се за съхранение на описание на схемата и друга информация като дизайнерски решения, потребителска информация и др.
 - **Active data dictionary** е достъпен от DBMS софтуер потребителите/DBA.
 - **Passive data dictionary** е достъпен само от потребителите/DBA.

Други Tools

- Application Development Environments and CASE (computer-aided software engineering) tools:
- Примери:
 - PowerBuilder (Sybase)
 - JBuilder (Borland)
 - JDeveloper 10G (Oracle)

Обичайните DBMS компоненти

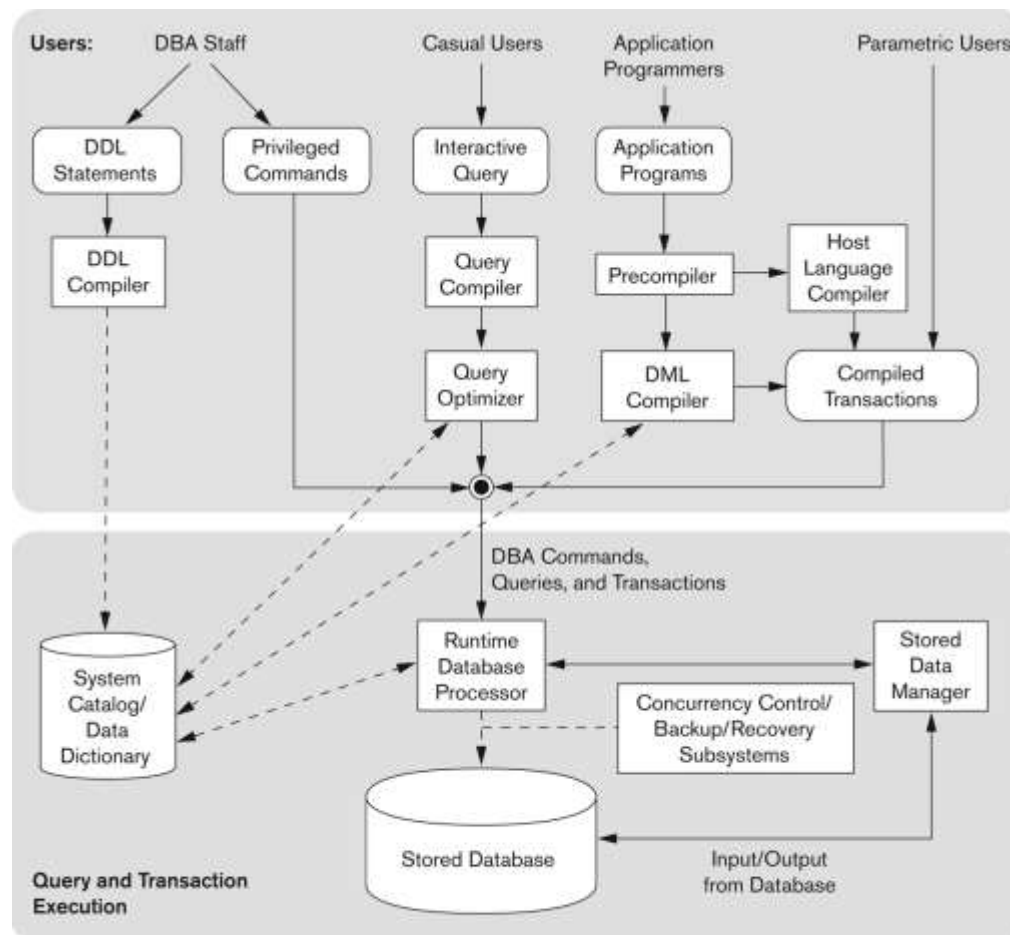


Figure 2.3

Component modules of a DBMS and their interactions.

Централизирани и Клиент/сървър DBMS архитектури

- Централизирани DBMS:
 - Комбинират всичко в една система - DBMS software, hardware, application programs и user interface processing software.
 - Потребителят може да се свърже през отдалечен терминал – всички обработки обаче са централизирани.

Физическа централизирана архитектура

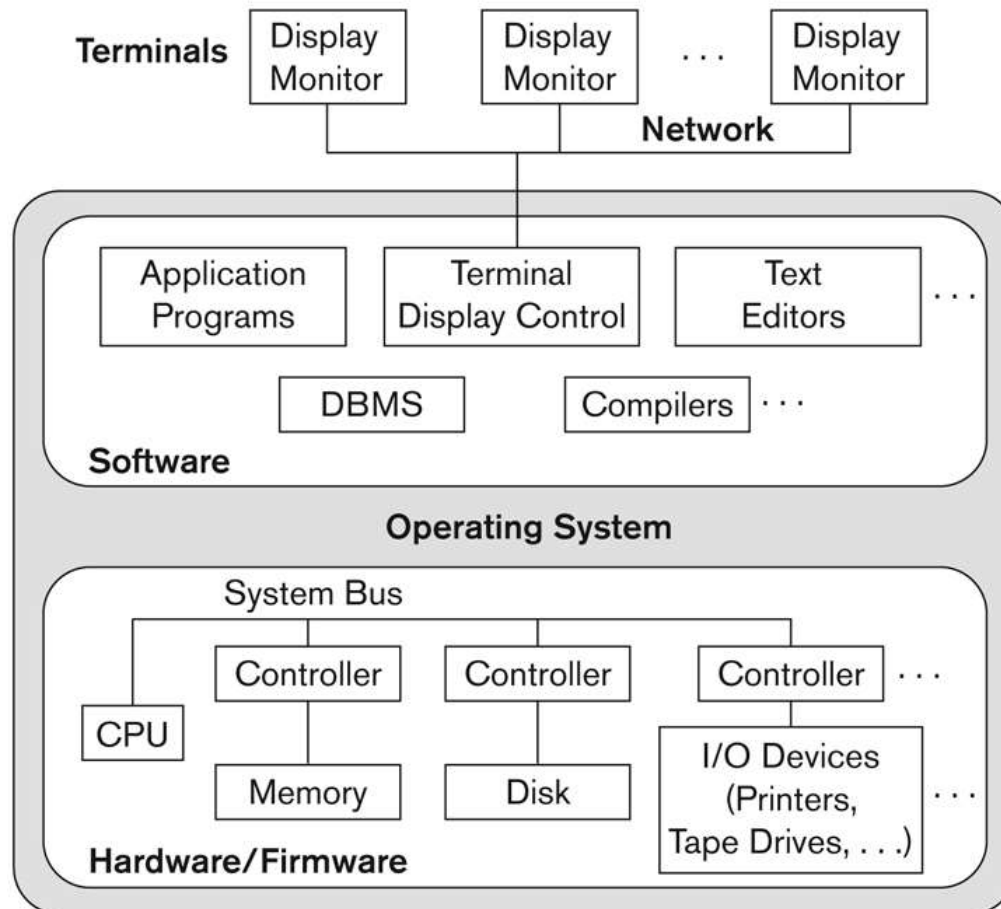


Figure 2.4

A physical centralized architecture.

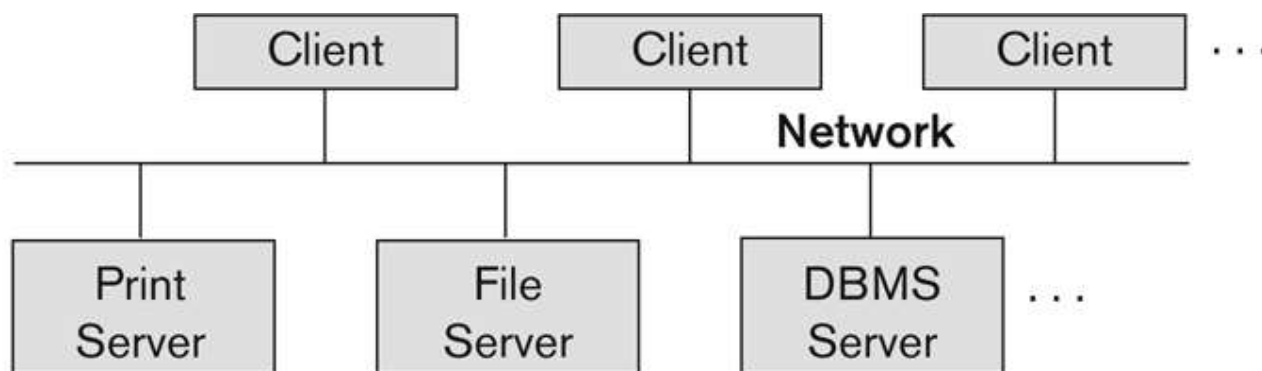
Основни 2-слойни клиент/сървър архитектури

- Специализирани сървъри със специализирани функции
 - Print server
 - File server
 - DBMS server
 - Web server
 - Email server
- Клиентите могат да достъпят сървъра при необходимост.

Логическа 2-слойна клиент/сървър архитектура

Figure 2.5

Logical two-tier
client/server
architecture.



Клиенти

- Предоставя подходящ интерфейс през клиентски софтуерен модул за достъп до различни сървърни ресурси.
- Клиентите могат да бъдат машини без дискове, РС-та или Workstations с инсталиран клиентски софтуер.
- Връзката към сървъра се осъществява през мрежа.
 - (LAN: local area network, wireless network, т.н.)

DBMS Server

- Предоставя услугите „запитвания“ и „транзакции“ към БД на клиента
- Relational DBMS servers се наричат и SQL servers, query servers или transaction servers
- Клиентските приложения използват Application Program Interface (**API**) за достъп до БД на сървъра през стандартен интерфейс:
 - ODBC: Open Database Connectivity standard
 - JDBC: за програмен достъп през Java
- Клиентът и сървърът трябва да инсталират подходящи клиентски и сървърен модули за ODBC или JDBC

2-слойна клиент/сървър архитектура

- Една клиентска програма може да се свърже към няколко DBMS, които се наричат data sources.
- В най-общия случай източник на данни може да бъде файл или друг non-DBMS софтуер, който управлява данни.
- Допускат се вариации в поведението на клиента, т.е. при някои DBMS обекти, се прехвърля повече функционалност (като речника на данните, оптимизация, recovery между множество сървъри и др.) към клиента.

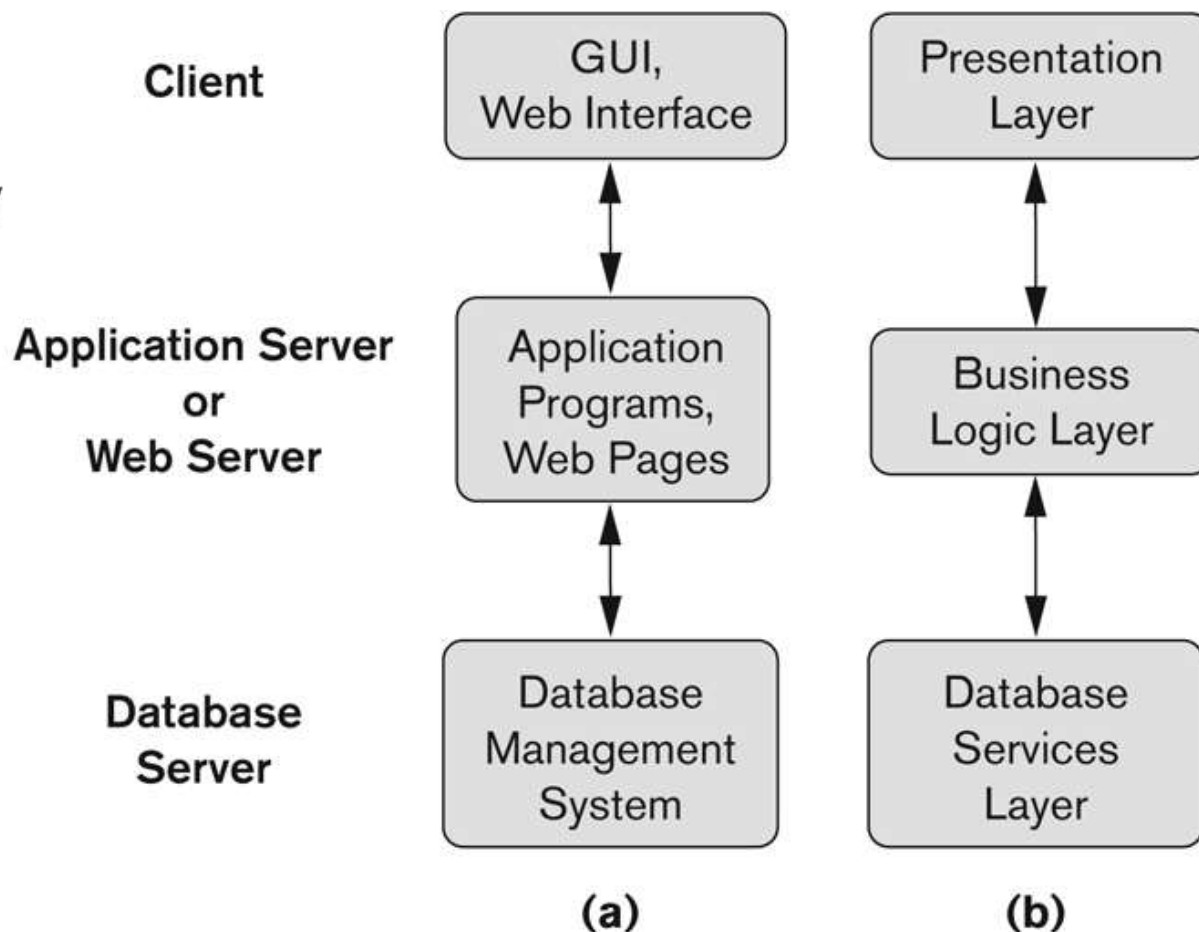
3-слойна клиент/сървър архитектура

- Използва се най-често при Web приложения
- Междинният слой се нарича Application Server или Web Server:
 - Съхранява софтуер за връзка в web и бизнес логиката на приложението. Използва се за достъп до данните от сБД сървър.
 - Работи като канал за изпращане на данни между сървъра и клиента на БД.
- 3-слойната архитектура подобрява сигурността чрез:
 - БД сървърът е достъпен само през междинния слой
 - Клиентът не може да достъпи директно БД сървъра

3-слойна клиент/сървър архитектура

Figure 2.7

Logical three-tier client/server architecture, with a couple of commonly used nomenclatures.



Класификация на DBMSs

- Според използвания модел на данните:
 - Традиционни: Relational, Network, Hierarchical.
 - Вградени: Object-oriented, Object-relational.
- Други класификации
 - Single-user (ползват се с PC)
срещу multi-user (повечето DBMSs).
 - Centralized (ползват един компютър с една БД)
срещу distributed (ползват множество компютри и множество БД)

Вариации на разпределени DBMSs (DDBMSs)

- Homogeneous DDBMS
- Heterogeneous DDBMS
- Federated or Multidatabase Systems
- Distributed Database Systems – днес са еквивалентни на client-server based database systems, защото:
 - Не поддържат напълно разпределена среда, а множество БД сървъри поддържат множество клиенти.

Ценово проучване на DBMSs

- Ценови обхват: от free open-source systems до конфигурации, струващи милиони долари
- Примери на free relational DBMSs: MySQL, PostgreSQL
- Комерсиалните DBMS предлагат допълнителни специализирани модули - time-series module, spatial data module, document module, XML module
 - Купуват се отделно
- Различни опции в лиценза: site license, max брой паралелни потребители към БД (seat license), single user и т.н.

История на данновите модели

- Network Model
- Hierarchical Model
- Relational Model
- Object-oriented Data Models
- Object-Relational Models

История на данновите модели

- **Network Model:**

- Първата мрежова DBMS е реализирана от Honeywell през 1964-65 (IDS System).
- Възприета е за тежка, поради поддръжката на CODASYL (Conference on Data Systems Languages) (CODASYL - DBTG report of 1971).
- По-късно е реализирана в различни големи системи - IDMS (Cullinet – cera Computer Associates), DMS 1100 (Unisys), IMAGE (H.P. (Hewlett-Packard)), VAX -DBMS (Digital Equipment Corp., next COMPAQ, now H.P.).

Пример на Network Model Schema

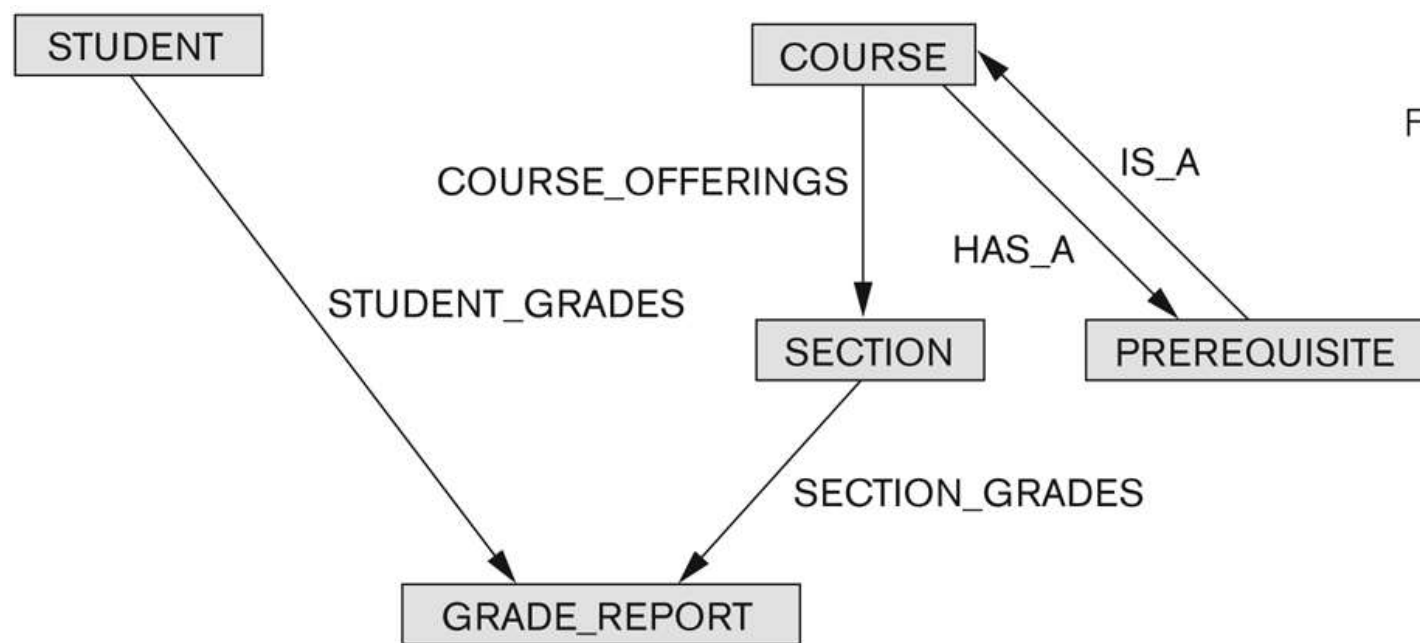


Figure 2.8

The schema of Figure 2.1 in network model notation.

Network Model

- Предимства:

- Мрежовият модел може да моделира сложни връзки и да представя семантика за добавяне/изтриване на връзки.
- Може да обхване повечето ситуации за моделиране чрез типове запис и типове връзки.
- Езикът е навигационен; ползва конструкции като FIND, FIND member, FIND owner, FIND NEXT within set, GET и др.
 - Достъпна е оптимална навигация през БД.

Network Model

- Недостатъци:
 - Навигационна и процедурна природа на обработките
 - БД съдържа сложен масив от указатели към записи.
 - Малки възможности за автоматична “query optimization”

История на данновите модели

- **Hierarchical Data Model:**

- Първоначално реализиран от обединените усилия на IBM и North American Rockwell около 1965. Като резултат е създадена IMS family of systems.
- Продуктът на IBM IMS е имал (и все още има) широка потребителска основа
- Йерархичният модел е формализиран на база IMS system
- Други системи, поддържащи този модел: System 2k (SAS inc.)

Hierarchical Model

- Предимства:
 - Лесен за създаване и поддържане
 - Съответства на определен брой естествено създадени йерархии като organization (“org”) chart
 - Езикът е лесен:
 - Ползва конструкции като GET, GET UNIQUE, GET NEXT, GET NEXT WITHIN PARENT и др.
- Недостатъци:
 - Навигационна и процедурна природа на обработките
 - БД се показва като линейна последователност от записи
 - Малки възможности за "query optimization"

История на данновите модели

- **Relational Model:**

- Предложен през 1970 от E.F. Codd (IBM), първата комерсиална система е създадена през 1981-82.
- Сега се ползва в множество комерсиални продукти (DB2, ORACLE, MS SQL Server, SYBASE, INFORMIX).
- Съществуват множество free open source реализации като MySQL, PostgreSQL
- Най-доминиращата, за разработка на приложения, БД.
- SQL relational standards: SQL-89 (SQL1), SQL-92 (SQL2), SQL-99, SQL3, ...
- Предмет на разглеждане в курса.

История на данновите модели

- **Object-oriented Data Models:**

- Множество модели са предложени за реализация в БД.
- Един компромисен модел на persistent O-O Programming Languages като C++ (в OBJECTSTORE или VERSANT), и Smalltalk (в GEMSTONE).
- В добавка системи като O2, ORION (в MCC – след това ITASCA), IRIS (в H.P.- ползват Open OODB).
- Object Database Standard: ODMG-93, ODMG-version 2.0, ODMG-version 3.0.

История на данновите модели

- **Object-Relational Models:**

- Най-новите разработки. Започват с Informix Universal Server.
- Релационните системи вграждат концепциите от обектните БД и се създават обектно-релационни БД.
- Като примери могат да се посочат късните версии на Oracle-10i, DB2 и SQL Server.
- Стандартите включват SQL-99 и се очаква разширение в бъдещите SQL стандарти.

Преглед

- Модели на данните и категориите им
- История на моделите на данни
- Схеми, инстанции и състояния
- Three-Schema Architecture
- Независимост на данните
- DBMS езици и интерфейси
- Database System Utilities and Tools
- Централизирани и клиент-сървър архитектури
- Класификация на DBMS